HCMC University Of Technology
Faculty of Computer Science & Engineering

Course: Computer network
Assignment 1 - Implement non-blocking HTTP server and chat application

March 7, 2026

Goal: The objective of this assignment is the application of major components in a computer network, for example, client server paradigm, the peer-to-peer paradigm and the network programming.

Content: In detail, student will practice with three major modules: the client-server HTTP communication, the peer-to-peer based chat application and the TCP/IP connection, which includes

- client processes and server processes.
- multiple peer processes connect together.
- the non-blocking network communication.

Besides, student will practice the design and implementation of Simple peer-to-peer protocol via TCP/IP.

Result: After this assignment, student can understand partly the principle of a computer network system. They can understand and design the role of each types of processes, i.e server process, client process and tracker peer process in a network communication.

Contents

1 Introduction **3**
1.1 An Overview . . . 3
1.2 Source Code . . . 4
1.3 Server Processes . . . 4
1.4 How to Create Server Processes? . . . 5
1.5 How to Run the Server . . . 5

2 Implementation **8**
2.1 Implement non-blocking mechanims . . . 8
2.2 Implement the authentication for HTTP server . . . 10
2.3 Implement hybrid chat application . . . 11
2.4 Put It All Together . . . 13

3 Submission **14**
3.1 Source code . . . 14
3.2 Requirements . . . 14
3.3 Grading . . . 14
3.4 Code of ethics . . . 14

1 Introduction

1.1 An Overview

In this work, students implement a non-blocking communication based HTTP server application equipped with a webapp chat application. The server application has multiple server process deployments with a proxy server process and multiple backend server process. An Asynchronous Application Routing ($\mathcal{A}$synap$\mathcal{R}$ous) framework deployment will help provide the environment for implementing peer-to-peer application. In this franework, the server framework allows multi-processes created by users to use non-blocking communicate mechanism.

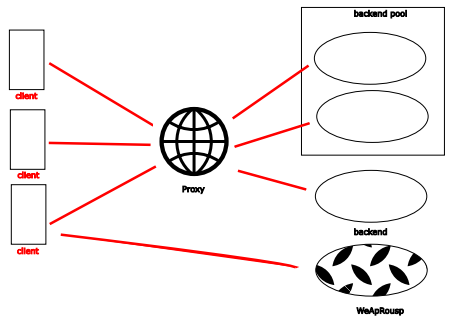

Figure 1: The general view of key modules in this assignment

An introduction to non-blocking communication Non-blocking TCP communication means the socket doesn't wait for operations to finish. Instead, recv() or send() return immediately, even if the buffer is full or empty, no data is available or only partial data is obtained. An error exception may occured like `BlockingIOError`. The blocking sockets pause your program until the operation finishes, while non-blocking sockets let your program continue and handle readiness explicitly.

An example Imagine a cashier who serves one customer at a time and refuses to look up until that customer is finished. If 100 customers arrive, you would need 100 cashiers.
Programming in blocking mode results in simpler code, but the program must wait as I/O operations are performed sequentially. Non-blocking mode runs in high performance servers, asynchronous I/O. It allows a single thread to handle thousands of concurrent connections efficiently. This design improves scalability, performance, and responsiveness compared to traditional blocking I/O models

Listing 1: TCP socket blocking vs non-blocking

```
# Blocking mode (default)
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("127.0.0.1", 888))

# This will WAIT until data sending complete
data = s.send(...)
print("Blocking recv:", data)

----
# Explicit switch to non-blocking mode
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
s.connect(("127.0.0.1", 8888))
```

```
s.setblocking(False)   # switch to non-blocking mode

try:
    data = s.send(...)   # returns immediately
    print("Non-blocking recv:", data)
except BlockingIOError:
    print("No data available yet")
```

1.2 Source Code

After downloading the source code of the assignment in the *Resource* section on the portal platform and extracting it, you will see the source code organized as follows.

- http_daemon
  - `daemon`: response.py, request.py, backend.py, httpadapter.py, asynaprous.py, dictionary.py
  - `www`: default location for static HTML webpages
    - index.html, login.html, form.html
  - `static`: default location for other HTML static objects
    - `images`: default location for images
    - `css`: default location for css file
    - `js`: default location for js
  - `apps`: default location for custom webapp
    - sampleapp.py
  - `cert`: default location for certification
  - `db`: default location for database
  - `config`: default location for proxy configuration file
  - `start_proxy.py`: the main program to start a proxy process
  - `start_backend.py`: the main program to start a backend process
  - `start_sampleapp.py`: the main program to start the asynchrnous webapp (AsynapRous) process

1.3 Server Processes

Proxy The proxy acts as an intermediary between the clients and the backend or webapp services. In this architecture, it receives incoming requests from multiple clients and forwards them to the appropriate backend resources.

Backend The backend refers to the core services or servers that handle the actual processing of client requests. The "previous step" proxy process routes the client requests to these backend units based on availability, load, or specific routing rules, ensuring efficient and reliable service delivery.

WebApp - AsynapRous The WebApp, labeled "AsynapRous" (short for Asynchronous App Routing) in the image, represents a specialized group of web-facing applications or services. These are typically user-facing components such as web servers or frontend applications that deliver content and interfaces to clients. The AsynapRous may be optimized for rendering HTML, serving static assets, or interacting with users directly through RESTful. Like the backend, it receives traffic through the proxy, which helps manage access and performance. AsynapRous can be run in independent mode without any proxy

1.4 How to Create Server Processes?

The content of each process is actually a copy of a program stored on disk. Thus to create a process, we must first generate the program which describes its content. A program is defined by a single file with the following format:

Listing 2: How to create a proxy

```
# From daemon/proxy.py
def create_proxy(ip, port, routes):
    run_proxy(ip, port, routes)
```

Listing 3: How to create a backend

```
# From daemon/backend.py
def create_backend(ip, port, routes={}):
    run_backend(ip, port, routes)
```

Listing 4: How to create a webapp

```
# From ./start_sampleapp.py
app = AsynapRous()

@app.route('/op1', methods=['POST'])
def op1(headers="guest", body="anonymous"):
    ...

@app.route('/op2', methods=['PUT'])
def op2(headers="guest", body="anonymous"):
    ...
```

1.5 How to Run the Server

What we are going to do in this assignment is to implement a simple HTTP server. To start these server process, we must create a script that revokes these server process and provide the hardware and the environment that we will work. The description file is defined in the following format (adapt these IP socket address to your system environment):

```
# From config/proxy.conf
host "192.168.56.103:8080" {
    proxy_pass http://192.168.56.103:9000;
}

host "app1.local" {
    proxy_pass http://192.168.56.103:9001;
}

host "app2.local" {
    proxy_set_header Host $host;

    proxy_pass http://192.168.56.210:9002;
    proxy_pass http://192.168.56.220:9002;

```

```
    dist_policy round-robin
}
```

The configuration basic structure Each configuration block begins with a host declaration followed by a set of instructions enclosed in

- host: declares the target hostname or IP address and optional port.
- Block {}: contains directives that define how requests to this host should be handled.

Supported directives

- `proxy_pass [URL];`: defines the backend server to which requests should be forwarded. You can specify multiple `proxy_pass` entries to enable load balancing policy.
- `proxy_set_header [HeaderName] [Value];`: sets custom headers for forwarded requests. This example alternate the original Host header field in the client requests.
- `dist_policy [strategy];` Specifies the distribution policy when multiple backends are defined.
  - round-robin: distributes requests evenly across all backends.
  - Other strategies (if supported) might include least-conn, random, etc.

Listing 5: "Start the processes"

```
#    Entry point for launching the server. This block parses command-line
#    arguments to determine the server's IP address and port.
#    :arg --server-ip (str): IP address to bind the server (default: 127.0.0.1).
#    :arg --server-port (int): Port number to bind the server (default: 9000).
    parser = argparse.ArgumentParser(
        prog='Proxy', description='', epilog='Proxy daemon'
    )
    parser.add_argument('--server-ip', default='0.0.0.0')
    parser.add_argument('--server-port', type=int, default=PROXY_PORT)

    args = parser.parse_args()
    ip = args.server_ip
    port = args.server_port

# From start_proxy.py
from daemon import create_proxy

routes = parse_virtual_hosts("config/proxy.conf")
create_proxy(ip, port, routes)

# From start_backend.py
from daemon import create_backend

create_backend(ip, port)

# From start_sampleapp.py
from daemon.asynaprous import AsynapRous

# Create a webapp instance
```

```
app = AsynapRous()

# Prepare and launch the RESTful application
app.prepare_address(ip, port)
app.run()
```

2 Implementation

2.1 Implement non-blocking mechanims

The implementation of non-blocking mechanisms relies on operating system services that allow I/O operations to be halted immediately rather than blocking execution. However, the responsibility for correctly managing ongoing communication and ensuring consistency is delegated to the application level. As a result, developers must design and implement appropriate handling logic to guarantee correctness and reliability in non-blocking communication system.
When a communication is halted, there are multiple processing flows to be handled.

- Multi-thread create a thread to handle the new created processing flows.
- Event-driven relies on registering callbacks that are executed when an operation completes. Instead of waiting for I/O, the program continues running, and the event loop triggers the callback once the communication is completed nad the data is ready. While powerful, it can lead to complex nested code structures, often referred to as "callback hell."
  - Some recently updated event mechansims (i.e futures or promises) allow chaining events by putting a placeholder for object representing a result that will be available later.
  - Event loop pub/sub puts a simple loop and use socket in a loop listens for events and dispatches them to handlers. Dive in the 'real' code and you will see the simple socket send()/recive() with some noisy wrappers.
- Coroutine (or asyncio, async/await) represents a more readable approach to asynchronous I/O. The **asyncio event loop** schedules and runs these coroutines, pausing them when they wait for I/O and resuming them once the operation completes.
  - **asyncio** is the framework for writing concurrent code using async/await syntax.
    - * `StreamReader` and `StreamWriter` are specific high-level APIs within `asyncio` used for handling I/O operations, i.e. network communication. They are components of the `asyncio`.
  - A loop `asyncio.run()` to start the event loop and run a coroutine until completion
  - The management mechanism asynchronous programs to runs your async tasks to completion and then clean up as in the AsyncIO runner [1].

Listing 6: AsyncIO runner

```
class Runner:
    def _lazy_init(self):
        # Initialize or obtain event loop
        if self._loop_factory is None:
            self._loop = events.new_event_loop()
...
        else:
            self._loop = self._loop_factory()

    def run(self, coro, *, context=None):
        """Run code in the embedded event loop."""
        self._lazy_init()
...
        task = self._loop.create_task(coro, context=context)
...
```

[1] https://github.com/python/cpython/blob/main/Lib/asyncio/runners.py

```
        # Threading main to signal control
        if threading.current_thread() is threading.main_thread()...
...
        # Run loop
        try:
            return self._loop.run_until_complete(task)

def run(main, *, debug=None, loop_factory=None):
    """Execute the coroutine and return the result.
    ...

    with Runner(debug=debug, loop_factory=loop_factory) as runner:
        return runner.run(main)
```

– It provides the AbstractEventLoop interface[2] and implement by BaseEventLoop in various specific Python versions The loop specifies methods for scheduling callbacks, running tasks, and handling delayed calls.

Listing 7: AsyncIO EventLoop

```
class BaseEventLoop(events.AbstractEventLoop):
    def run_forever(self):
        try:
            while True:
                self._run_once()
        ...


    def _run_once(self):
        """Run one full iteration of the event loop.
        ...
        event_list = self._selector.select(timeout)
        self._process_events(event_list)
        ...
```

Table 1: List of non-blocking mechanism

Mechanism	**Types**
Multi-thread	Thread based
Callback	Event-driven
Asyncio	Co-routine
Future / Promise	Event-driven, allow chaining
Pub/Sub Event Loop	Event-driven

Non-blocking Javascript backend framework JavaScript are interested in asynchronous communication on the web framework. It supports sending a request to a server and continue running other code while it waits for the response. This approach makes web pages feel faster and more responsive.

[2]https://github.com/python/cpython/blob/main/Lib/asyncio/events.py
[2] https://github.com/python/cpython/blob/3.14/Lib/asyncio/base_events.py

Node.js vs Bun Node.js started as a single-threaded runtime that relied heavily on callbacks, and later added coroutine-like features through async/await. Bun, by contrast, is designed with multi-threading at its core, which can deliver higher performance but also requires more management effort and system resources.

Node.js vs Nest.js Nest is a Backend Server Framework which can be used to build up powerful APIs. It provides a structured architecture that makes applications easier to develop and maintain, it components convenient for plugging into applications. Core Components of NestJS includes Modules,Controller, Providers (Services, business logic); the group of Decorators: @Module(), @Controller(), @Injectable(), @Get(), @Post(); Middleware: Injectable, NestMiddleware.

Express, Fastify Express provides APIs using a callback-based approach for Node.js, and its architecture has remained largely unchanged over time. Fastify, on the other hand, supports coroutines directly through async/await, allowing developers to write asynchronous code without requiring special error-handling.

Important note React, Next.js,... are front-end frameworks. Our focus is not on creating beautiful user interfaces. Course CO3093/CO3094 is communication-centric, not interface-centric. Textbox, button and label are sufficient to develop a communication program. We are more interested in efficiently sending bytes to millions of user connections than refining the interface. We do not encourage framework usages; instead, we encourage you to develop your own framework using the provided non-blocking mechanisms.

Framework	**Types**
Express	Callback
Fastify	Callback + coroutine
NestJs	Coroutine (async/await)
NodeJs	Callback (legacy) and coroutine (recent)

Provided non-blocking inter-daemon connection This module implements the inter-daemon incoming connection handling by accepting socket connections and delegating each client to the HttpAdapter through one of three non-blocking strategies: multi-threading, callback/event-driven using selectors, or coroutine-based async/await, selected via the mode_async flag. The design allows the backend to flexibly adapt to different concurrency models, ensuring that developers can experiment with traditional multi-thread approaches, event-driven callbacks, or coroutine depending on the performance and scalability requirements of their application. We have provided supporting of the non-blocking connection in:

- inter-daemon communication
- RESTful API handler

2.2 Implement the authentication for HTTP server

Description To authenticate a user, there are two common approaches include HTTP headers and Cookies. The former RFC 2617 or the updated version rfc 7235 with extended proxy usecase, specifies HTTP use the header "WWW-Authenticate" to request for an authentication; then, the browser responds with an "Authorization" header that carries user login details. The latter RFC 6265, uses cookies to store with "Set-Cookie" header when a user log in successfully. The server sends back a instruction that stores a small piece of data in user browser. From then on, The browser automatically includes that cookie in every request, so the server knows it's still that logged-in user.
Notes it is recommended to use incognito tab since HTTP apply conditional GET by default. The browser cache can be cleared to perform the system experiment properly.

Task requirements Implement the authentication mechanism. Refers the RFCs for the technical details.

- RFC 2617 - HTTP Authentication: Basic and Digest Access Authentication
  - * https://www.rfc-editor.org/rfc/rfc2617
- RFC 7235 - Hypertext Transfer Protocol (HTTP/1.1): Authentication
  - * https://www.rfc-editor.org/rfc/rfc7235
- RFC 6265 - HTTP State Management Mechanism (Cookies)
  - * https://www.rfc-editor.org/rfc/rfc6265

2.3 Implement hybrid chat application

Description Develop a hybrid network application that has a chat system (similar to skype), combining both client-server and peer-to-peer (P2P) paradigms. The application must support channel management, and synchronization across distributed peers.
Equipped webapp - AsynapRous The webapp support RESTful (Representational State Transfer) is an architectural style for designing networked applications. It relies on standard HTTP methods [GET], [POST], [PUT], [DELETE] to perform operations on resources, which are typically represented as URLs.
RESTful APIs are stateless, scalable, and easy to integrate across platforms. Custom route URLs allow developers to define meaningful and readable endpoints. These routes improve clarity and make your API friendly to handle more complex logic including query parameters or nested paths.
The provided framework also support 3 types of non-continous communication multi-thread approaches, event-driven callbacks, or coroutine.

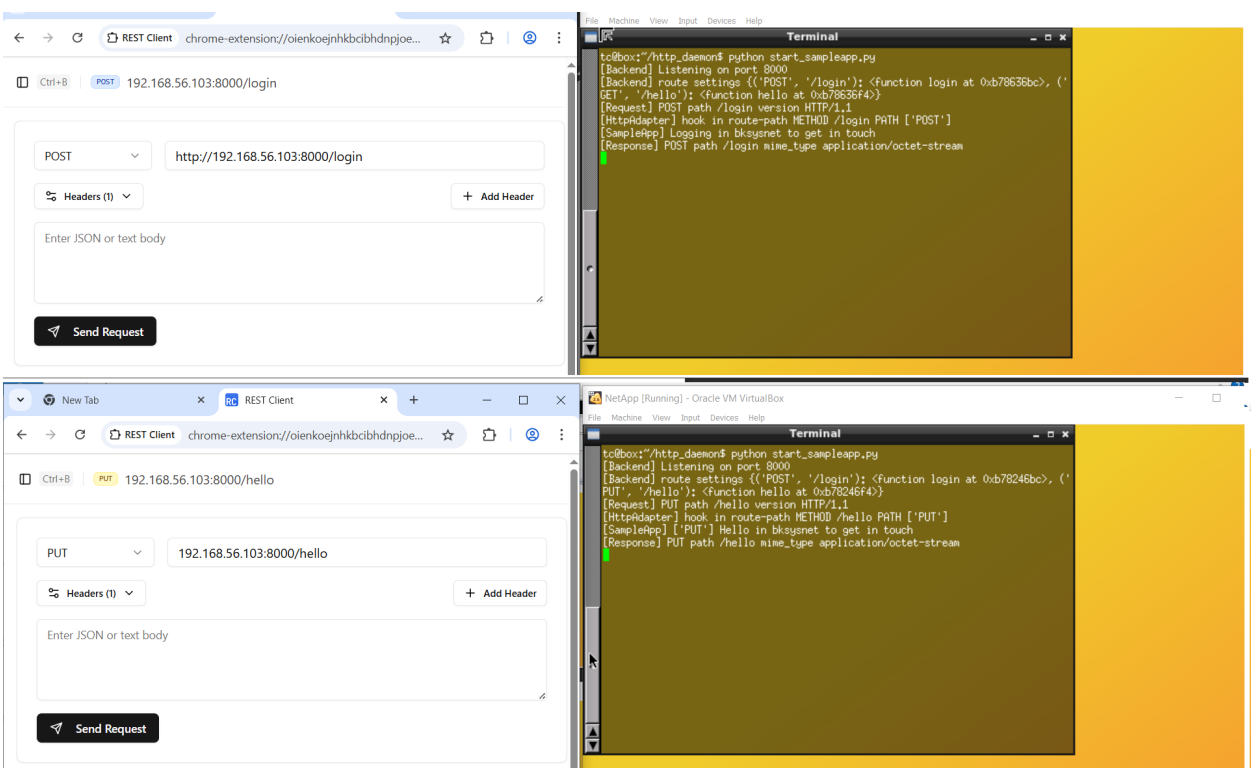

Figure 2: The screenshot of webapp (AsynapRous) serves request at `[PUT]/login` and `[POST]/hello]`

In Figure 2, a user webapp is started at port `http://IP:8000` with two handler at method [PUT] path "/login" and method [POST] path "/hello". The handler is installed by user webapp and is pres

Core functional requirements the application has these following requirements
Initialization phase (Client-Server paradigm)

- *Peer registration:* When a new peer joins, it must submit its IP and port to the centralized server.
- *Tracker update:* The centralized server must maintain a tracking list of active peers.
- *Peer discovery*: Peers can request the current list of active peers from the server.
- *Connection setup:* Peers use the tracking list to initiate direct P2P connections.

Peer chatting phase (Peer-to-Peer Paradigm)

- *Broadcast connection:* A peer must broadcast messages to all connected peers.
- *Direct peer communication:* Peers exchange messages without routing through the centralized server during live sessions.

Channel management

- *Channel listing:* Users can view channels they've joined.
- *Message display:* Each channel has a scrollable message window.
- *Message submission:* UI must support text input and submission.
- *No edit/delete:* Messages are immutable once sent.
- *Notification system:* Users must be notified when new messages arrive.
- *Access control (Optional):* Channels may define custom access policies.

Task requirements

- Non-blocking communication among multi-daemons. Student can based on callback or coroutine for each daemon in each peer to communicate with other peers.
  - Javascript is only allowed in client-side to perform user update or GUI interaction asynchronously.
  - The backend must be implemented using the standard Python library, and the use of JavaScript or existing web framework is strictly forbidden.
- Client-server process programming
- Protocol design in message communication and the processing procedure
  - **login API** example `http://IP:port/login/`
  - **submit-info API** example `http://IP:port/submit-info/`
  - **add-list API** example `http://IP:port/add-list/`
  - **get-list API** example `http://IP:port/get-list/`
  - **connect-peer API** example `http://IP:port/connect-peer/`
  - **broadcast-peer API** example `http://IP:port/broadcast-peer/`
  - **send-peer API** example `http://IP:port/send-peer/`
- Concurrency
- Error Handling

2.4 Put It All Together

Finally, we combine both authentication work and AsynapRous derivationvs to form a complete server application.

3 Submission

3.1 Source code

Requirement: you have to code the HTTP server and application followed by the coding style. References:

- PEP 8 – Style Guide for Python Code - https://peps.python.org/pep-0008 and
- PEP 257 – Docstring Conventions - https://peps.python.org/pep-0257/

3.2 Requirements

Application and protocol design implement the design based on non-blocking protocol is mandatory. The framework must be built by yourself.

> **Notice** Continuous data flows or streaming processing may result in data being skipped or truncated if not managed properly, so students are encouraged to consult their lab teacher for guidance in selecting the most suitable mechanism for their interests and requirements. Stream processing will be deferred and addressed in a future task as part of the HTTPv3 implementation.

Authentication handling implement the authentication handling and the cookies access control subsystem2.2.

Chat application implement the hybrid chat application.
After you finish the assignment, move your report to source code directory and compress the entire directory into a single file named `assignment_STUDENTID.zip` and submit to LMS.

3.3 Grading

You must complete this assignment in groups of 4 or 5 students. The overall grade for your group is determined by the following components:

- Demonstration (7 points)
  - Authentication: 2 points
  - ChatApp Client-server paradigm: 1 points
  - ChatApp Peer-to-peer paradigm: 2 points
  - Non-blocking communication mechanism: 2 points
- Report (3 points)

3.4 Code of ethics

Faculty staff members involved in code development reserved all the copyright of the project source code.
Source Code License Grant: Author(s) hereby grant(s) to Licensee personal permission to use and modify the Licensed Source Code for the sole purpose of studying while attending the course CO3093/CO3094 at HCMUT.

Revision History

Revision	Date	Author(s)	Description
1.0	10.2025	pdnguyen,	Initialize HTTP server, WeApRous Webapp framework
2.0	03.2026	pdnguyen,	Add non-blocking network communication

daemon.asynaprous

index
daemon/asynaprous.py

```
deamon.asynaprous
~~~~~~~~~~~~~~~~~

This module provides a AsynapRous object to deploy RESTful url web app with routing
```

Modules

asyncio inspect

Classes

builtins.object
    AsynapRous

class **AsynapRous**(builtins.object)

```
The fully mutable :class:`AsynapRous <AsynapRous>` object, which is a lightweight,
mutable web application router for deploying RESTful URL endpoints.

The `AsynapRous` class provides a decorator-based routing system for building simple
RESTful web applications.  The class allows developers to register route handlers
using decorators and launch a TCP-based backend server to serve RESTful requests.
Each route is mapped to a handler function based on HTTP method and path. It mappings
supports tracking the combined HTTP methods and path route mappings internally.

Usage::
  >>> import daemon.asynaprous
  >>> app = AsynapRous()
  >>> @app.route('/login', methods=['POST'])
  >>> def login(headers="guest", body="anonymous"):
  >>>     return {'message': 'Logged in'}

  >>> @app.route('/hello', methods=['GET'])
  >>> def hello(headers, body):
  >>>     return {'message': 'Hello, world!'}

  >>> app.run()
```

Methods defined here:

__init__(self)

```
Initialize a new AsynapRous instance.

Sets up an empty route registry and prepares placeholders for IP and port.
```

prepare_address(self, ip, port)

```
Configure the IP address and port for the backend server.

:param ip (str): The IP address to bind the server.
:param port (str): The port number to listen on.
```

route(self, path, methods=['GET'])

```
Decorator to register a route handler for a specific path and HTTP methods.

:param path (str): The URL path to route.
:param methods (list): A list of HTTP methods (e.g., ['GET', 'POST']) to bind.

:rtype: function - A decorator that registers the handler function.
```

run(self)

```
Start the backend server and begin handling requests.

This method launches the TCP server using the configured IP and port,
and dispatches incoming requests to the registered route handlers.

:raise: Error if IP or port has not been configured.
```

Data descriptors defined here:

__dict__

```
dictionary for instance variables (if defined)
```

__weakref__

```
list of weak references to the object (if defined)
```

daemon.proxy

index
daemon/proxy.py

```
daemon.proxy
~~~~~~~~~~~~~~~~~

This module implements a simple proxy server using Python's socket and threading libraries.
It routes incoming HTTP requests to backend services based on hostname mappings and returns
the corresponding responses to clients.

Requirement:
-----------------
- socket: provides socket networking interface.
- threading: enables concurrent client handling via threads.
- response: customized :class: `Response <Response>` utilities.
- httpadapter: :class: `HttpAdapter <HttpAdapter >` adapter for HTTP request processing.
- dictionary: :class: `CaseInsensitiveDict <CaseInsensitiveDict>` for managing headers and cookies.
```

Modules

datetime	os	threading
mimetypes	socket	

Functions

create_proxy(ip, port, routes)

```
Entry point for launching the proxy server.

:params ip (str): IP address to bind the proxy server.
:params port (int): port number to listen on.
:params routes (dict): dictionary mapping hostnames and location.
```

forward_request(host, port, request)

```
Forwards an HTTP request to a backend server and retrieves the response.

:params host (str): IP address of the backend server.
:params port (int): port number of the backend server.
:params request (str): incoming HTTP request.

:rtype bytes: Raw HTTP response from the backend server. If the connection
              fails, returns a 404 Not Found response.
```

handle_client(ip, port, conn, addr, routes)

```
Handles an individual client connection by parsing the request,
determining the target backend, and forwarding the request.

The handler extracts the Host header from the request to
matches the hostname against known routes. In the matching
condition,it forwards the request to the appropriate backend.

The handler sends the backend response back to the client or
returns 404 if the hostname is unreachable or is not recognized.

:params ip (str): IP address of the proxy server.
:params port (int): port number of the proxy server.
:params conn (socket.socket): client connection socket.
:params addr (tuple): client address (IP, port).
:params routes (dict): dictionary mapping hostnames and location.
```

resolve_routing_policy(hostname, routes)

```
Handles an routing policy to return the matching proxy_pass.
It determines the target backend to forward the request to.
```

```
:params host (str): IP address of the request target server.
:params port (int): port number of the request target server.
:params routes (dict): dictionary mapping hostnames and location.
```

run_proxy(ip, port, routes)

```
Starts the proxy server and listens for incoming connections.

The process dinds the proxy server to the specified IP and port.
In each incomping connection, it accepts the connections and
spawns a new thread for each client using `handle_client`.


:params ip (str): IP address to bind the proxy server.
:params port (int): port number to listen on.
:params routes (dict): dictionary mapping hostnames and location.
```

Data

BASE_DIR = ''
PROXY_PASS = {'192.168.56.103:8080': ('192.168.56.103', 9000), 'app1.local': ('192.168.56.103', 9001), 'app2.local': ('192.168.56.103', 9002)}

daemon.backend

index
daemon/backend.py

```
daemon.backend
~~~~~~~~~~~~~~~~~

This module provides a backend object to manage and persist backend daemon.
It implements a basic backend server using Python's socket and threading libraries.
It supports handling multiple client connections concurrently and routing requests using a
custom HTTP adapter.

Requirements:
--------------
- socket: provide socket networking interface.
- threading: Enables concurrent client handling via threads.
- response: response utilities.
- httpadapter: the class for handling HTTP requests.
- CaseInsensitiveDict: provides dictionary for managing headers or routes.


Notes:
------
- The server create daemon threads for client handling.
- The current implementation error handling is minimal, socket errors are printed to the console.
- The actual request processing is delegated to the HttpAdapter class.

Usage Example:
--------------
>>> create_backend("127.0.0.1", 9000, routes={})
```

Modules

argparse	inspect	selectors
asyncio	mimetypes	socket
datetime	os	threading

Functions

async **async_server**(ip='0.0.0.0', port=7000, routes={})

create_backend(ip, port, routes={})

```
Entry point for creating and running the backend server.

:param ip (str): IP address to bind the server.
:param port (int): Port number to listen on.
:param routes (dict, optional): Dictionary of route handlers. Defaults to empty dict.
```

handle_client(ip, port, conn, addr, routes)

```
Initializes an HttpAdapter instance and delegates the client handling logic to it.

:param ip (str): IP address of the server.
:param port (int): Port number the server is listening on.
:param conn (socket.socket): Client connection socket.
:param addr (tuple): client address (IP, port).
:param routes (dict): Dictionary of route handlers.
```

handle_client_callback(server, ip, port, conn, addr, routes)

```
Initialize connection instance and delegates the client handling logic to it.

:param ip (str): IP address of the server.
:param port (int): Port number the server is listening on.
:param routes (dict): Dictionary of route handlers.
```

async **handle_client_coroutine**(reader, writer)

```
Coroutine in async communication to initialize connection instance
then delegates the client handling logic to it.

:param reader (StreamReader): Stream reader wrapper.
:param write (Stream write): Stream write wrapper.
```

run_backend(ip, port, routes)

```
Starts the backend server, binds to the specified IP and port, and listens for incoming
connections. Each connection is handled in a separate thread. The backend accepts incoming
connections and spawns a thread for each client.


:param ip (str): IP address to bind the server.
:param port (int): Port number to listen on.
:param routes (dict): Dictionary of route handlers.
```

Data

BASE_DIR = ''
mode_async = 'coroutine'
sel = <selectors.EpollSelector object>

daemon.httpadapter

index
daemon/httpadapter.py

```
daemon.httpadapter
~~~~~~~~~~~~~~~~~

This module provides a http adapter object to manage and persist
http settings (headers, bodies). The adapter supports both
raw URL paths and RESTful route definitions, and integrates with
Request and Response objects to handle client-server communication.
```

Modules

asyncio inspect

Classes

builtins.object
    HttpAdapter

class **HttpAdapter**(builtins.object)

```
HttpAdapter(ip, port, conn, connaddr, routes)

A mutable :class:`HTTP adapter <HTTP adapter>` for managing client connections
and routing requests.

The `HttpAdapter` class encapsulates the logic for receiving HTTP requests,
dispatching them to appropriate route handlers, and constructing responses.
It supports RESTful routing via hooks and integrates with :class:`Request <Request>`
and :class:`Response <Response>` objects for full request lifecycle management.

Attributes:
    ip (str): IP address of the client.
    port (int): Port number of the client.
    conn (socket): Active socket connection.
    connaddr (tuple): Address of the connected client.
    routes (dict): Mapping of route paths to handler functions.
    request (Request): Request object for parsing incoming data.
    response (Response): Response object for building and sending replies.
```

Methods defined here:

__init__(self, ip, port, conn, connaddr, routes)

```
Initialize a new HttpAdapter instance.

:param ip (str): IP address of the client.
:param port (int): Port number of the client.
:param conn (socket): Active socket connection.
:param connaddr (tuple): Address of the connected client.
:param routes (dict): Mapping of route paths to handler functions.
```

add_headers(self, request)

```
Add headers to the request.

This method is intended to be overridden by subclasses to inject
custom headers. It does nothing by default.


:param request: :class:`Request <Request>` to add headers to.
```

build_json_response(self, req, resp)

```
Builds a :class:`Response <Response>` object from JSON data

:param req: The :class:`Request <Request>` used to generate the response.
:param resp: The  response object.
:rtype: Response
```

build_proxy_headers(self, proxy)

```
Returns a dictionary of the headers to add to any request sent
through a proxy.

:class:`HttpAdapter <HttpAdapter>`.

:param proxy: The url of the proxy being used for this request.
:rtype: dict
```

build_response(self, req, resp)

```
Builds a :class:`Response <Response>` object

:param req: The :class:`Request <Request>` used to generate the response.
:param resp: The  response object.
:rtype: Response
```

handle_client(self, conn, addr, routes)

```
Handle an incoming client connection.

This method reads the request from the socket, prepares the request object,
invokes the appropriate route handler if available, builds the response,
and sends it back to the client.

:param conn (socket): The client socket connection.
:param addr (tuple): The client's address.
:param routes (dict): The route mapping for dispatching requests.
```

async **handle_client_coroutine**(self, reader, writer)

```
Handle an incoming client connection using stream reader writer asynchronously.

This method reads the request from the socket, prepares the request object,
invokes the appropriate route handler if available, builds the response,
and sends it back to the client.

:param conn (socket): The client socket connection.
:param addr (tuple): The client's address.
:param routes (dict): The route mapping for dispatching requests.
```

Readonly properties defined here:

extract_cookies

```
Build cookies from the :class:`Request <Request>` headers.

:param req:(Request) The :class:`Request <Request>` object.
:param resp: (Response) The res:class:`Response <Response>` object.
:rtype: cookies - A dictionary of cookie key-value pairs.
```

Data descriptors defined here:

__dict__

```
dictionary for instance variables (if defined)
```

__weakref__

```
list of weak references to the object (if defined)
```

Data and other attributes defined here:

__attrs__ = ['ip', 'port', 'conn', 'connaddr', 'routes', 'request', 'response']

daemon.request

index
daemon/request.py

```
daemon.request
~~~~~~~~~~~~~~~~~

This module provides a Request object to manage and persist
request settings (cookies, auth, proxies).
```

Classes

builtins.object
    Request

class **Request**(builtins.object)

```
The fully mutable "class" `Request <Request>` object,
containing the exact bytes that will be sent to the server.

Instances are generated from a "class" `Request <Request>` object, and
should not be instantiated manually; doing so may produce undesirable
effects.

Usage::

  >>> import deamon.request
  >>> req = request.Request()
  ## Incoming message obtain aka. incoming_msg
  >>> r = req.prepare(incoming_msg)
  >>> r
  <Request>
```

Methods defined here:

__init__(self)
```
Initialize self.  See help(type(self)) for accurate signature.
```

extract_request_line(self, request)

fetch_headers_body(self, request)
```
Prepares the given HTTP headers.
```

prepare(self, request, routes=None)
```
Prepares the entire request with the given parameters.
```

prepare_auth(self, auth, url='')

prepare_body(self, data, files, json=None)

prepare_content_length(self, body)

prepare_cookies(self, cookies)

prepare_headers(self, request)
```
Prepares the given HTTP headers.
```

Data descriptors defined here:

__dict__
```
dictionary for instance variables (if defined)
```

__weakref__

```
list of weak references to the object (if defined)
```

Data and other attributes defined here:

__attrs__ = ['method', 'url', 'headers', 'body', '_raw_headers', '_raw_body', 'reason', 'cookies', 'body', 'routes', 'hook']

daemon.response

index
daemon/response.py

```
daemon.response
~~~~~~~~~~~~~~~~

This module provides a :class: `Response <Response>` object to manage and persist
response settings (cookies, auth, proxies), and to construct HTTP responses
based on incoming requests.

The current version supports MIME type detection, content loading and header formatting
```

Modules

datetime	mimetypes	os

Classes

builtins.object
    Response

class **Response**(builtins.object)

```
Response(request=None)

The :class:`Response <Response>` object, which contains a
server's response to an HTTP request.

Instances are generated from a :class:`Request <Request>` object, and
should not be instantiated manually; doing so may produce undesirable
effects.

:class:`Response <Response>` object encapsulates headers, content,
status code, cookies, and metadata related to the request-response cycle.
It is used to construct and serve HTTP responses in a custom web server.

:attrs status_code (int): HTTP status code (e.g., 200, 404).
:attrs headers (dict): dictionary of response headers.
:attrs url (str): url of the response.
:attrsencoding (str): encoding used for decoding response content.
:attrs history (list): list of previous Response objects (for redirects).
:attrs reason (str): textual reason for the status code (e.g., "OK", "Not Found").
:attrs cookies (CaseInsensitiveDict): response cookies.
:attrs elapsed (datetime.timedelta): time taken to complete the request.
:attrs request (PreparedRequest): the original request object.

Usage::

  >>> import Response
  >>> resp = Response()
  >>> resp.build_response(req)
  >>> resp
  <Response>
```

Methods defined here:

__init__(self, request=None)

```
Initializes a new :class:`Response <Response>` object.

: params request : The originating request object.
```

build_content(self, path, base_dir)

```
Loads the objects file from storage space.

:params path (str): relative path to the file.
:params base_dir (str): base directory where the file is located.

:rtype tuple: (int, bytes) representing content length and content data.
```

build_notfound(self)

```
Constructs a standard 404 Not Found HTTP response.

:rtype bytes: Encoded 404 response.
```

build_response(self, request, envelop_content=None)

```
Builds a full HTTP response including headers and content based on the request.

:params request (class:`Request <Request>`): incoming request object.

:rtype bytes: complete HTTP response using prepared headers and content.
```

build_response_header(self, request)

```
Constructs the HTTP response headers based on the class:`Request <Request>
and internal attributes.

:params request (class:`Request <Request>`): incoming request object.

:rtypes bytes: encoded HTTP response header.
```

get_mime_type(self, path)

```
Determines the MIME type of a file based on its path.

"params path (str): Path to the file.

:rtype str: MIME type string (e.g., 'text/html', 'image/png').
```

prepare_content_type(self, mime_type='text/html')

```
Prepares the Content-Type header and determines the base directory
for serving the file based on its MIME type.

:params mime_type (str): MIME type of the requested resource.

:rtype str: Base directory path for locating the resource.

:raises ValueError: If the MIME type is unsupported.
```

Data descriptors defined here:

__dict__

```
dictionary for instance variables (if defined)
```

__weakref__

```
list of weak references to the object (if defined)
```

Data and other attributes defined here:

__attrs__ = ['_content', '_header', 'status_code', 'method', 'headers', 'url', 'history', 'encoding', 'reason', 'cookies', 'elapsed', 'request', 'body', 'reason']

Data

BASE_DIR = ''

daemon.dictionary

index
daemon/dictionary.py


```
# Copyright (C) 2026 pdnguyen of HCMC University of Technology VNU-HCM.
# All rights reserved.
# This file is part of the CO3093/CO3094 course.
#
# AsynapRous release
#
# The authors hereby grant to Licensee personal permission to use
# and modify the Licensed Source Code for the sole purpose of studying
# while attending the course
#
```


Classes

collections.abc.MutableMapping(collections.abc.Mapping)
CaseInsensitiveDict

class **CaseInsensitiveDict**(collections.abc.MutableMapping)

```
CaseInsensitiveDict(*args, **kwargs)

The :class:`CaseInsensitiveDict<MutableMapping>` object, which
contains a custom behavior of MutuableMapping.

Usage::

  >>> import tools
  >>> word = CaseInsensitiveDict(status_code='404', msg="Not found")
  >>> code = word['status_code']
  >>> code
  404

  >>> msg = word['msg']
  >>> s.send(r)
  Not found

  >>> print(word)
  {'status_code': '404', 'msg': 'Not found'}
```

Method resolution order:
- CaseInsensitiveDict
- collections.abc.MutableMapping
- collections.abc.Mapping
- collections.abc.Collection
- collections.abc.Sized
- collections.abc.Iterable
- collections.abc.Container
- builtins.object

Methods defined here:

__delitem__(self, key)

__getitem__(self, key)

__init__(self, *args, **kwargs)

```
Initialize self.  See help(type(self)) for accurate signature.
```

__iter__(self)

__len__(self)

__setitem__(self, key, value)

Data descriptors defined here:

__dict__
```
dictionary for instance variables (if defined)
```

__weakref__
```
list of weak references to the object (if defined)
```

Data and other attributes defined here:

__abstractmethods__ = frozenset()

Methods inherited from collections.abc.MutableMapping:

clear(self)
```
D.clear() -> None.  Remove all items from D.
```

pop(self, key, default=<object object at 0x7f8b9c6483b0>)
```
D.pop(k[,d]) -> v, remove specified key and return the corresponding value.
If key is not found, d is returned if given, otherwise KeyError is raised.
```

popitem(self)
```
D.popitem() -> (k, v), remove and return some (key, value) pair
as a 2-tuple; but raise KeyError if D is empty.
```

setdefault(self, key, default=None)
```
D.setdefault(k[,d]) -> D.get(k,d), also set D[k]=d if k not in D
```

update(self, other=(), /, **kwds)
```
D.update([E, ]**F) -> None.  Update D from mapping/iterable E and F.
If E present and has a .keys() method, does:     for k in E: D[k] = E[k]
If E present and lacks .keys() method, does:     for (k, v) in E: D[k] = v
In either case, this is followed by: for k, v in F.items(): D[k] = v
```

Methods inherited from collections.abc.Mapping:

__contains__(self, key)

__eq__(self, other)
```
Return self==value.
```

get(self, key, default=None)
```
D.get(k[,d]) -> D[k] if k in D, else d.  d defaults to None.
```

items(self)
```
D.items() -> a set-like object providing a view on D's items
```

keys(self)
```
D.keys() -> a set-like object providing a view on D's keys
```

values(self)
```
D.values() -> an object providing a view on D's values
```

Data and other attributes inherited from collections.abc.Mapping:

__hash__ = None

__reversed__ = None

Class methods inherited from collections.abc.Collection:

__subclasshook__(C) from abc.ABCMeta

```
Abstract classes can override this to customize issubclass().

This is invoked early on by abc.ABCMeta.__subclasscheck__().
It should return True, False or NotImplemented.  If it returns
NotImplemented, the normal algorithm is used.  Otherwise, it
overrides the normal algorithm (and the outcome is cached).
```

Class methods inherited from collections.abc.Iterable:

__class_getitem__ = GenericAlias(...) from abc.ABCMeta

```
Represent a PEP 585 generic type

E.g. for t = list[int], t.__origin__ is list and t.__args__ is (int,).
```

Data

__warningregistry__ = {'version': 0}

daemon.utils

index
daemon/utils.py

```
# Copyright (C) 2026 pdnguyen of HCMC University of Technology VNU-HCM.
# All rights reserved.
# This file is part of the CO3093/CO3094 course.
#
# AsynapRous release
#
# The authors hereby grant to Licensee personal permission to use
# and modify the Licensed Source Code for the sole purpose of studying
# while attending the course
#
```

Functions

get_auth_from_url(url)

```
Given a url with authentication components, extract them into a tuple of
username,password.

:rtype: (str,str)
```

apps.sampleapp

index
apps/sampleapp.py

```
app.sampleapp
~~~~~~~~~~~~~~~~
```

Modules

importlib	json	os	sys

Functions

create_sampleapp(ip, port)

Data

app = <daemon.asynaprous.AsynapRous object>

start_backend


```
start_backend
~~~~~~~~~~~~~~~~~

This module provides a simple entry point for deploying backend server process
using the socket framework. It parses command-line arguments to configure the
server's IP address and port, and then launches the backend server.
```

Modules

argparse	socket

Data

PORT = 9000

start_proxy


```
start_proxy
~~~~~~~~~~~~~~~~~

This module serves as the entry point for launching a proxy server using Python's socket framework.
It parses command-line arguments to configure the server's IP address and port, reads virtual host
definitions from a configuration file, and initializes the proxy server with routing information.

Requirements:
--------------
- socket: provide socket networking interface.
- threading: enables concurrent client handling via threads.
- argparse: parses command-line arguments for server configuration.
- re: used for regular expression matching in configuration parsing
- response: response utilities.
- httpadapter: the class for handling HTTP requests.
- urlparse: parses URLs to extract host and port information.
- daemon.create_proxy: initializes and starts the proxy server.
```

Modules

argparse	re	socket	threading

Functions

parse_virtual_hosts(config_file)

```
Parses virtual host blocks from a config file.

:config_file (str): Path to the NGINX config file.
:rtype list of dict: Each dict contains 'listen'and 'server_name'.
```

Data

PROXY_PORT = 8080

start_sampleapp


```
start_sampleapp
~~~~~~~~~~~~~~~~~

This module provides a sample RESTful web application using the AsynapRous framework.

It defines basic route handlers and launches a TCP-based backend server to serve
HTTP requests. The application includes a login endpoint and a greeting endpoint,
and can be configured via command-line arguments.
```

Modules

argparse	json	socket

Data

PORT = 2026